

Progetto Database a Grafo: Piattaforma di Formazione

Introduzione

Il presente documento descrive il lavoro svolto per la progettazione, l'implementazione e la validazione di un knowledge graph in Neo4j a supporto di una piattaforma di formazione online. L'obiettivo del progetto è rappresentare corsi, docenti, studenti, argomenti e certificati in una struttura a grafo capace di esplicitare le relazioni tra le entità, ricostruire percorsi di apprendimento basati su prerequisiti e individuare corsi o studenti connessi da interessi comuni — scenari che un modello relazionale tradizionale esprimerebbe con maggiore difficoltà.

1. Modello a Grafo

La progettazione è partita dall'analisi dei requisiti del dominio formativo, da cui sono state individuate cinque entità principali, modellate come nodi, e le relazioni che le collegano.

Nodi e proprietà principali:

- *Corso*: id (chiave), titolo, livello, durata_ore, categoria;
- *Docente*: id (chiave), nome, cognome, email, specializzazione;
- *Studente*: matricola (chiave), nome, cognome, email, data_iscrizione;
- *Argomento*: id (chiave), nome, descrizione;
- *Certificato*: id (chiave), data_rilascio, punteggio, codice_verifica.

Relazioni principali:

- (Docente) -[TIENE] -> (Corso), con proprietà dal;
- (Studente) -[ISCRITTO_A] -> (Corso), con proprietà data_iscrizione, stato;
- (Studente) -[OTTIENE] -> (Certificato);
- (Certificato) -[CERTIFICA] -> (Corso);
- (Corso) -[TRATTA] -> (Argomento);
- (Argomento) -[PREREQUISITO_DI] -> (Argomento), relazione ricorsiva tra nodi dello stesso tipo.

Due scelte di modellazione meritano una nota di progettazione specifica. Il Certificato è stato rappresentato come nodo autonomo, e non come semplice proprietà dello Studente o del Corso, perché possiede attributi propri (data di rilascio, punteggio, codice di verifica) e partecipa a due relazioni distinte. La matricola, inoltre, è stata scelta come chiave identificativa dello Studente.

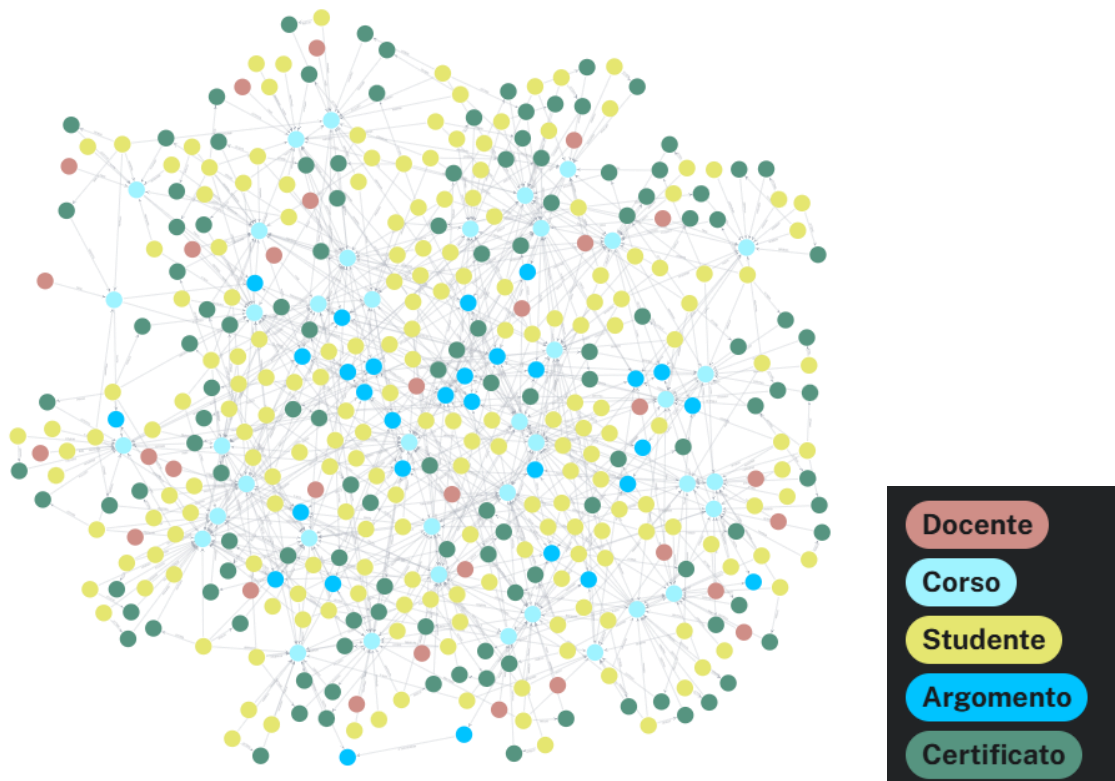
2. Implementazione in Neo4j

L'infrastruttura del grafo è stata sviluppata in ambiente Neo4j, con creazione dello schema e popolamento interamente tramite script Cypher (Schema_e_Popolamento.txt).

Il popolamento è stato generato con il supporto del LLM Claude, calibrato per produrre volumi realistici e distribuzioni non banali:

- 40 Corsi, distribuiti su 7 categorie tematiche e 3 livelli di difficoltà (Base, Intermedio, Avanzato);
- 30 Docenti, con copertura garantita: l'algoritmo di assegnazione assicura che ogni docente tenga almeno un corso;
- 200 Studenti, ciascuno iscritto a 1-4 corsi con stato differenziato (completato, in corso, abbandonato);
- 30 Argomenti, definiti manualmente per garantire coerenza didattica nelle catene di prerequisiti;
- Certificati generati dinamicamente dalle sole iscrizioni con stato "completato" (circa 100-150 record).

La grande mole di dati rende difficile la visualizzazione del grafo. Si rimanda ad alcuni file nel repository (argomenti_prerequisiti.png, studenti_iscritti_a.png, argomenti_trattati_da_corsi.png) per una visione più dettagliata di alcune relazioni.



3. Vincoli e Integrità dei Dati

Il sistema impone l'integrità dei dati su più livelli.

- **A livello dichiarativo:** constraint di unicità su ciascuna entità (Corso.id, Docente.id, Studente.matricola, Argomento.id, Certificato.id) e indici su Corso.categoria e Studente.email per velocizzare i pattern di ricerca più frequenti;
- **A livello logico/procedurale:** Neo4j non impone nativamente regole di dominio complesse, pertanto sono state predisposte query di verifica dedicate (Query_esplorative_Graph_DB.txt) per controllare: assenza di docenti privi di corsi, assenza di corsi privi di argomenti trattati, assenza di cicli nella catena dei prerequisiti, coerenza tra stato dell'iscrizione e presenza del certificato, e validità dei range numerici (durata, punteggio, date non future);

4. Query di Analisi

Nel corso dello sviluppo sono state formulate quindici query di esplorazione, pensate per coprire in modo esaustivo gli scenari gestionali e didattici della piattaforma. Questo set completo costituisce il file di test (Query_esplorative_Graph_DB.txt) del progetto.

Di queste quindici, sei query sono state selezionate e integrate nello strato applicativo Python (neo4j_queries.py), in quanto rappresentative delle esigenze operative ricorrenti della piattaforma e sufficienti a dimostrare l'intero flusso di interazione, sia in Cypher standard sia tramite la Graph Data Science Library (GDS). Tali query sono:

- 1) *Corsi collegati a un argomento:* individua tutti i corsi che trattano un determinato argomento, direttamente o tramite catene di prerequisiti. Permette di rispondere al requisito di esplorazione delle relazioni tra corsi.

- 2) *Corsi consigliati per uno studente*: suggerisce corsi non ancora seguiti il cui argomento è uno sviluppo naturale di ciò che lo studente ha già completato, individuando percorsi di apprendimento personalizzati.
- 3) *Tasso di completamento per categoria*: aggrega le iscrizioni per categoria di corso, calcolando la percentuale di completamento come indicatore amministrativo generale.
- 4) *Studenti simili (GDS – Node Similarity)*: individua studenti connessi da interessi comuni sulla base dei corsi condivisi, tramite l'algoritmo di similarità Jaccard della Graph Data Science Library, più scalabile su grafi grandi rispetto a un doppio MATCH manuale.
- 5) *Argomenti centrali (GDS – PageRank)*: individua gli argomenti più "fondamentali" nella rete di prerequisiti, utile per la progettazione del curriculum e per stabilire l'ordine didattico ottimale.
- 6) *Comunità di corsi (GDS – Louvain)*: rileva raggruppamenti naturali di corsi affini in base agli argomenti condivisi, anche oltre le categorie assegnate manualmente in fase di popolamento.

Le restanti nove query del set completo coprono controlli e analisi aggiuntive, quali il carico di lavoro dei docenti, i corsi mai completati da alcuno studente, gli argomenti privi di collegamenti di prerequisito, la distribuzione dei corsi per livello e la retention degli studenti iscritti di recente.

5. Conclusione

Il progetto ha portato alla realizzazione di un knowledge graph completo per la gestione di una piattaforma di formazione, dalla progettazione concettuale del modello a nodi e relazioni fino alla verifica sistematica della sua affidabilità su un dataset di dimensioni realistiche.

Le scelte progettuali riflettono esigenze concrete del dominio didattico: l'assegnazione dei corsi garantisce che nessun docente resti privo di corsi, la rappresentazione del Certificato come nodo autonomo consente di interrogarne direttamente gli attributi, e l'uso della matricola come chiave dello Studente lega l'identificativo tecnico a un dato riconoscibile nel dominio reale. Il ricorso alla Graph Data Science Library, infine, ha permesso di superare i limiti delle query Cypher pure nell'individuazione di similarità, centralità e comunità nel grafo, aspetti che un motore puramente relazionale gestirebbe con maggiore difficoltà e minore naturalezza espressiva.